

tf.dynamic_partition Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/dynamic_partition

Partitions data into num_partitions tensors using indices from partitions.

Function Signatures

```
tf.dynamic_partition( data: Annotated[Any,  
TV_DynamicPartition_T], partitions: Annotated[Any,  
_atypes.Int32], num_partitions: int, name=None )
```

Code Examples

```
tf.dynamic_partition(  
data: Annotated[Any, TV_DynamicPartition_T],  
partitions: Annotated[Any, _atypes.Int32],  
num_partitions: int,  
name=None  
)
```

```
tf.dynamic_partition(  
    data: Annotated[Any, TV_DynamicPartition_T],  
    partitions: Annotated[Any, _atypes.Int32],  
    num_partitions: int,  
    name=None  
)
```

```
outputs[i].shape = [sum(partitions == i)] +  
    data.shape[partitions.ndim:]  
outputs[i] = pack([data[js, ...] for js if partitions[js] == i])
```

```
outputs[i].shape = [sum(partitions == i)] +  
    data.shape[partitions.ndim:]  
outputs[i] = pack([data[js, ...] for js if partitions[js] == i])
```

```
# Scalar partitions.  
partitions = 1  
num_partitions = 2  
data = [10, 20]  
outputs[0] = [] # Empty with shape [0, 2]  
outputs[1] = [[10, 20]]  
# Vector partitions.  
partitions = [0, 0, 1, 1, 0]  
num_partitions = 2  
data = [10, 20, 30, 40, 50]  
outputs[0] = [10, 20, 50]  
outputs[1] = [30, 40]
```

```
# Scalar partitions.
partitions = 1
num_partitions = 2
data = [10, 20]
outputs[0] = [] # Empty with shape [0, 2]
outputs[1] = [[10, 20]]
# Vector partitions.
partitions = [0, 0, 1, 1, 0]
num_partitions = 2
data = [10, 20, 30, 40, 50]
outputs[0] = [10, 20, 50]
outputs[1] = [30, 40]
```

Documentation

Partitions data into num_partitions tensors using indices from partitions.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.dynamic_partition`

For each index tuple `js` of size `partitions.ndim`, the slice `data[js, ...]` becomes part of `outputs[partitions[js]]`. The slices with `partitions[js] = i` are placed in `outputs[i]` in lexicographic order of `js`, and the first dimension of `outputs[i]` is the number of entries in `partitions` equal to `i`. In detail,

`data.shape` must start with `partitions.shape`.

For example:

See `dynamic_stitch` for an example on how to merge partitions back.

Raises

- `InvalidArgumentError` in following cases:
 - If `partitions` is not in range `[0, num_partitions)`
 - If `partitions.shape` does not match prefix of `data.shape` argument.

Returns